Extending SISO to Multiple Subjects

Aditya Ramesh
Carnegie Mellon University
adityar2@andrew.cmu.edu

Gorkem Yar
Carnegie Mellon University
gyar@andrew.cmu.edu

Bhavesh Sood
Carnegie Mellon University
bsood@andrew.cmu.edu

Abstract

Recent work in image generation has led to training-free personalization methods requiring only one reference image. However, there remains work towards extending these image personalization pipelines to multi-subject examples. Single Image Subject Optimization (SISO) is a fine-tuning method that enables generative models to produce personalized generated images from one reference example. In this project, we propose to extend SISO by enabling multi-subject personalization and comparing performance with different parameter efficient finetuning methods.

1 Introduction

Current research in personalized image generation has made considerable progress for singular subject image generation through parameter efficient fine-tuning (PEFT) methods [9] [17]. These methods append additional vector embeddings, LoRA [3] modules, and cross attention [1]. A promising method in this space is *Single Image Subject Optimization* (SISO) [17], which improved on previous methods by requiring only one reference image to create high-fidelity outputs that preserve subject identity across a variety of prompts through incorporating similarity-based loss functions that utilizes the embedding space of DINO [2] and IR [10].

While SISO demonstrates impressive results for single-subject personalization, it is not suited to model scenes involving multiple personalized subjects. Certain applications require multiple reference subject in the personalized image without sacrificing image quality or subject fidelity.

We propose an extension to the SISO framework that supports **multi-subject personalization**. Our goal is to independently optimize LoRA modules for each subject and integrate them into a shared generative pipeline, enabling coherent scene generation from prompts involving multiple known entities. Additionally, we aim to explore and benchmark this approach across multiple diffusion backbones—such as SDXL-Turbo [11], SANA [16], and Stable Diffusion 1.5 [8]—to assess efficiency, scalability, and output quality. Through this exploration, we hope to contribute to the development of lightweight, adaptable personalization pipelines that can scale to complex generative tasks while maintaining high visual fidelity.

2 Dataset, Task

2.1 Dataset

We use the subject images taken from DreamBooth [9]. For subject-driven image generation, we will use the setup comprising of a simple prompt for finetuning a reference subject image. The dataset comprises of 30 unique subjects, including a mix of inanimate objects like plushie, clock, toys and live entities such as pets. Specifically, the dataset contains 21 object categories (e.g., backpacks, sunglasses, stuffed animals) and 9 live subjects (e.g., cats, dogs, and cartoon characters). Each subject is represented by a single reference image while finetuning, aligning with the goals of single-image personalization. Each subject has around 4-6 images, giving us a total of 158 images. All data used in the project was either collected by the authors of DreamBooth [9] or sourced from publicly available datasets such as Unsplash[1]. For multi-subject we plan to use sets of two subjects from within these 30 subjects only.

[1] https://unsplash.com

10623 Class Project (2025).

2.2 Task

SISO is a finetuning method to enable personalization of generated image by applying a loss function that compares the generated image to the reference image and tunes the model to produce more images with similar subject [17]. The loss function is defined as the weighted sum between the distance in DINO [2] and IR [10] embedding space, where the weights are hyperparameters.

During the optimization process, a baseline image is produced with a non-descriptive prompt (ex: "an image of a dog"). The loss function is applied and the model generates another image, repeating this process until some stopping criteria is achieved. This iterative process may result in multiple iterations before the model is fully tuned to the reference image. At this point, the model can then be used for more complicated and unseen prompts (ex: "an image of a dog in outer space"), enabling zero-shot performance with the reference subject. In addition, multi-subject generation has approaches modifying the embedding and cross-attention inputs as seen in FastComposer [14].

Our task is to incorporate this pipeline for multi-subject image generation. A few methods are proposed below to accomplish this:

- Incorporate multiple images during fine-tuning to simultaneously learn multiple subjects.
- Hot swapping multiple tuned LoRA modules on each reference image with masks to edit images with reference subjects.

To evaluate performance, the DINO and IR cosine similarities of the reference images and generated images after fine-tuning will be compared between models. In addition, naturalness will be evaluated using FID, KID, and CMMD, where lower values represent closer alignment to real image distributions. By observing these metrics, multi-subject personalization could be implemented while retaining identity fidelity and realism.

3 Related Work

Personalized image generation has been approached through different techniques, including computing vision embeddings from a reference image to inject into the diffusion process and fine-tuning methods on generative models. Popular SOTA methods include IP-Adapter [18], which implements a decoupled cross-attention mechanism to efficiently condition image generation on a reference image. Another method, InstantID [12], improves on this by implementing a unique ID embedding, an image adapter, and a modified ControlNet [19] to take advantage of distinguishing features in human faces. DreamBooth [9] re-purposes rarely used text embeddings with the features and semantics of the reference photos. ELITE [13] also follows a similar approach by inverting reference photos into textual embeddings, significantly decreasing the generation time compared to previous implementations. Other methods such as PhotoMaker [5] exist which has the capability to combine multiple subject's features into one realistic subject, allowing for more novel applications of image personalization. These methods involve some training process which may be computationally expensive for certain projects but retain high quality personalized images.

Less work has been done with multi-subject personalization. *Imagine Yourself* inject concatenated reference photos into the cross attention layer [1]. Nested attention layers represents the subject to a single text token through an encoder-decoder model with a nested set of attention layers [7]. Other methods utilizes methods found in many VLMs, where images are embedded into vectors through an encoder of a VAE before being concatenated with text that specify which subject to include in the generated image [15].

4 Approach

In this section, the SISO (Single Image Subject Optimization) model will be explained detailly. Then our potential multi-subject support will be explained. In addition to this, after the implementation of the multi subject support diffusion backbones (pretrained) models will be examined to find the best modeling architecture.

4.1 Baseline: SISO

SISO injects a randomly initialized LoRA module into a frozen Stable Diffusion XL (SDXL) backbone (speciffically to Unet Layer) The optimization process is guided by a two similarity losses that enforce the generated image to resemble the reference subject and prompt both semantically and perceptually. The semantic similarity is enforced using embeddings from DINO, which capture high-level object structure, while perceptual similarity is ensured through image retrieval (IR) features that emphasize fine-grained visual details and identity.

- **Input:** A reference (subject) image x_{ref} and a simple prompt such as "a photo of [V] dog". For the prompt, CLIP encoders and CLIP tokenizers are used generate embeddings from the prompt.

- **Generation:** The LoRA-injected pipeline generates an image x_{gen} from the prompt.

- **Similarity Loss:** DINO and IR feature extractors compute semantic and identity-level embeddings.

$$\mathcal{L}_{sim}(x_{gen}, x_{ref}) = \alpha \cdot \delta_{DINO}(x_{gen}, x_{ref}) + \beta \cdot \delta_{IR}(x_{gen}, x_{ref})$$

  where δ is the cosine distance between feature vectors, and α, β are weighting parameters typically set to 1.0.

All generation occurs in the latent space, where the model denoises a learned latent representation instead of operating directly in pixel space. These latents are then decoded into RGB images using a VAE. To ensure identity preservation, the generated image is compared against the reference image using DINO Vision Transformer and an IR model. The resulting loss signal is backpropagated to update only the LoRA parameters during inference-time optimization.
The original SISO codebase can be found here. We replicated the SISO baseline and re-implemented SISO, In the experiment section, we will share the available results.

4.2 Proposed Approach: Multi-Subject Masked Algorithm

To extend the SISO framework to support the generation of images involving multiple subjects (e.g., "a [V1] cat playing with a [V2] dog"), we first experimented with a masking-based approach. We used single LoRA layer like the SISO. However, instead of optimizing the weights for a single subject we tried optimizizing the weights for two subjects simultaneously.

New Loss Function

Given reference images x^A_{ref} and x^B_{ref} for two subjects, we compute a total similarity loss as a weighted sum of per-subject similarity terms:

$$\mathcal{L}_{total} = \mathcal{L}_{sim}(x_{gen}, x^A_{ref}, m_A) + \mathcal{L}_{sim}(x_{gen}, x^B_{ref}, m_B) \tag{1}$$

- x_{gen} is the generated image at the current optimization step.

- m_A, m_B are binary masks isolating the regions corresponding to subject A and subject B, respectively.

- $\mathcal{L}_{sim}$ denotes the same combined DINO + IR embedding loss used in SISO.

With the help of the masks we compared each generated subject to its reference independently. So, the losses generated for each subject will activate another weight in LoRA layers.

Subject-Aware Masking: To obtain spatial masks, we applied a **pretrained object detection and segmentation model** to the generated images. Specifically, we used **Grounding DINO** [6] to detect bounding box that surrounds the subject on the generated image. Later, we fed the bounding box and generated image into **SAM (Segment Anything Model)** [4] to convert each box into a high-resolution segmentation mask. This enables subject-specific losses to backpropagate only through relevant regions. However, in practice, we found that GroundingDINO often struggled to produce accurate bounding boxes for multi-object scenes, which led to incorrect masks and poor supervision. In these cases, masks collapsed to one dominant subject, leading to degenerate behavior and loss of identity in the secondary subject.

4.3 Proposed Approach: Multi-Subject Alternate Training

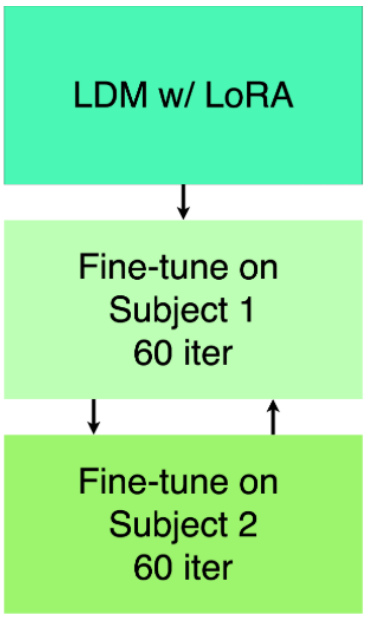

Figure 1: Sequential optimization steps for each subject.

While the masking-based approach allowed for subject-specific loss application, we observed that inaccurate masks often caused identity degradation, especially when one subject visually dominated the scene. To address this, we developed an alternative strategy that trains on each subject sequentially in an alternating round-robin fashion, removing the need for object segmentation.

In this method, we optimize a shared LoRA layer using multiple subject-specific training rounds, each focusing on only one subject at a time. During each round, we present a simple prompt (e.g., "a photo of a [V]") along with the reference image. We compute the DINO + IR similarity loss and backpropagate only for that subject. Once the loss drops below a defined threshold or a maximum number of steps is reached, we switch to the next subject and repeat the process.

At inference, the final LoRA module has jointly learned features for both subjects and can generate coherent images from prompts involving both entities (e.g., "a [V1] sitting next to a [V2]"). We found this approach to be more stable, less GPU-intensive (Since it does not use segmentation models), and more consistent in preserving identity compared to the masked method. However, it is important to note that this model suffers when both of the subjects are complex like dog and cat. This method generally gives better results when one of the subjects are simpler like teapot or backpack.

5 Experiments

In Table 1, we compare subject-driven generation methods across two main evaluation axes: **identity preservation** and **naturalness**. Our baseline, **SISO (baseline)**, is a single-subject optimization method described in the previous sections.

We introduce two other models that also does subject driven image generation, AttnDreamBooth and ClassDiffusion.

- **SISO (ours)**: Our implementation version of single-subject SISO pipeline (baseline).
- **Multiple Subject Model**: A generation model trained to handle multiple subjects simultaneously with Stable Diffusion XL (**SDXL**) as the backbone.

The following experiment was conducted on the basis of the DreamBooth dataset.

	Identity Preservation		Naturalness		
Method	**DINO↑**	**IR↑**	**FID↓**	**KID↓**	**LPIPS↓**
AttnDreamBooth	0.47	0.51	164.4	0.004	-
ClassDiffusion	0.50	0.59	166.6	0.003	-
TIGIC	0.51	0.58	143.3	0.0066	0.22
SwapAnything	0.45	0.60	185.7	0.0277	**0.11**
SISO (baseline)	0.48	0.53	**149.2**	**0.002**	0.18
SISO (ours)	0.29	0.40	245.8	0.004	0.737
Multiple Subject SISO (ours)	**0.65**	**0.76**	217.1	0.05	0.71

Table 1: Comparison of identity preservation and naturalness across methods.

We reimplemented the SISO pipeline *from scratch*, where specific examples from our implementation can be found in the Appendix. We used Fréchet inception distance (FID) and Kernel Inception Distance (KID) as measure of the generated image's fidelity. These are measures of the similarity between the distribution of the reference images and generated images, where a lower number indicates more similarity between the generated images. It also acts as a measure of the fidelity of the images generated. Learned Perceptual Image Patch Similarity (LPIPS) is a

measure of the perceptual similarity between the two distributions. Similarly to FID and KID, a lower number indicates more similarity between the two distributions.

Our results during training show similar image fidelity to the paper's implementations with comparable loss during training. We then implemented the proposed approach and tested over a number of combination of classes, listed in the Appendix. To calculate DINO and IR loss, we take the losses from the last epoch of fine-tuning. To calculate FID, KID, and LPIPS, we generated two images for each combination of classes then used the reference subject images and the generated images to calculate the scores.

Our result in Table 1 show our model's performance compared to the papers and other image personalization models. The other model's results were taken from the paper, where LPIPS metric was missing for DreamBooth [9] and ClassDiffusion. Our reimplementation of SISO, despite showing visual fidelity and similar functionality, showed lower performance than the paper's implementation. This could be explained by different stopping criteria and resolution size, primarily motivated by limited hardware resources. Our multi-subject extension also showed lower performance, but this is expected as the reference image's now contain multiple subjects, so we expect a lower score over the single subject personalization models. We found higher DINO and IR score on our model however, leading us to believe the model extends well into the multi-subject domain. Images generated with multi-subject SISO show high fidelity and realism, indicating our implementation works. We find if similar subjects are used, the features are blended, resulting in one morphed subject. For example, a black cat with an orange cat may generate mixed cat and not our two distinct cats.

While training on a 40GB A100, we found it takes 50 seconds to fine-tune SDXL-Turbo on a single reference photo for 50 iterations. Once the model is tuned, each subsequent generated image takes 1.69 seconds.

6 Code Overview

We re-implemented the SISO pipeline to easily edit it for our use. Fig 10 shows the code snippet that added LoRA modules to the model using HuggingFace's libraries. We chose to include the modules to the U-Net's attention mechanism, including the projection of key, query, and value vectors, to remain consistent with the original implementation of SISO. In addition, we utilized the torchmetrics library to calculate LPIPS, FID, and KID score shown in Fig 11 where we compare the two subject images along with two generated images of "subject 1 next to subject 2". These functions calculated the values found within Table 1.

7 Timeline

Table 2 below summarizes the time our team spent on various stages of the project. It includes reading prior work, implementing both baseline and proposed methods, running experiments, and preparing the final deliverables such as the poster and executive summary.

Task	**Hours Spent**	**Description**
Reading papers & prior work (SISO, DINO, etc.)	12 hrs	Reviewing literature including SISO, DreamBooth, multi-subject personalization, and DINO/IR losses.
Project Proposal Writing	8 hrs	Drafting proposal and formatting in LaTeX.
Dataset curation & setup	3 hrs	Downloading DreamBooth data, pairing subjects for multi-subject combinations.
Re-implementing SISO baseline	10 hrs	Injecting LoRA, prompt conditioning, replicating DINO+IR loss logic.
Midway Executive Summary Writing	12 hrs	Drafting Midway summary sections, formatting in LaTeX, and inserting images.
Implementing Masked Loss (multi-subject v1)	12 hrs	Integrating GroundingDINO + SAM for mask extraction; computing loss per subject.
Troubleshooting Masking Failures	4 hrs	Analyzing GroundingDINO misdetections and mask collapse failures.
Implementing Alternate Training	4 hrs	Optimizing each subject independently in round-robin style and tuning.
Experimentation & result collection	10 hrs	Running different methods, saving images, computing DINO/IR, FID metrics.
Poster design	5 hrs	Visual layout, selecting images and results, preparing short pitch.
Final Report Writing	12 hrs	Drafting executive summary sections, formatting in LaTeX, inserting images.
Total	**92 hrs**	

Table 2: Final time-tracked breakdown of team contributions by task.

8 Research Log

We started off by understanding the paper SISO [17] and its code. The authors provided an implementation spread across multiple files, using various arguments to run the code. Additionally, weights for certain models like the IR model weren't directly included in the codebase; instead, a link was provided, so we had to integrate them manually. Since we were working with SDXL [11], which requires more GPU RAM than our local machines could provide, we shifted to an A100 40GB GPU instance. We began re-implementing the code ourselves in a notebook, going over the paper and attempting to recreate a pipeline similar to the authors' implementation by loading the Hugging Face models directly instead of using the authors' functions. We faced some trouble integrating the IR weights but were eventually able to do so. At this point, we were successfully able to call the main SDXL model with the LoRA PEFT method injected and run diffusion over a sample noise and prompt.

Next, we implemented the algorithm proposed by SISO [17], enabling us to pass a subject image, run diffusion with cross-attention on it, compute the IR and DINO loss between the decoded image and the subject image, and backpropagate the loss to the LoRA weights. We then downloaded the dataset and wrote a script to load it cleanly. The generated images started to resemble the subject, but they were still far from what the paper demonstrated. This led us to spend several hours tuning hyperparameters — adjusting denoising steps, changing the LoRA rank, learning rate, and the number of fine-tuning iterations on the subject.

Finally, we were able to get results that looked like this:

Figure 2: The subject image is shown on the left, and the generated images are on the right. We illustrate the progression of the generated image during fine-tuning using the simple prompt "image of a {class_name}", optimized with IR and DINO loss over the subject image, across iterations [0, 2, 5, 8, 10, 30, 50].

Now we shifted our focus to trying multi-subject optimization using a single image of each subject. First, we attempted the approach using the loss in Equation 1. To do this, we used Grounding DINO [6] to obtain the bounding boxes of the generated images at each step and then used SAM [4] to extract the mask for each subject. These masks would be used to compute the IR and DINO losses for each subject separately.

However, after several attempts, we realized that Grounding DINO was not able to accurately find the bounding boxes of the separated subjects—it was returning only the bounding box of the larger subject. You can checkout an example in Appendix 9.

So here, we had to pivot from our approach of using masks and instead decided to use the basic method by which neural networks learn from multiple examples—iterating over them. Thus, we tried iterating over two subjects one at a time, allowing the model to learn both subjects. We also increased the LoRA weights by raising the rank, hoping that more weights would help the model capture both subjects better. Next, we trained on one subject for a certain number of iterations and then switched to the other subject, treating it like single-subject fine-tuning but with separate prompts. We then tested by generating images using a prompt containing both subjects. The results weren't always as good as those from the single-subject setup, but we were glad to achieve some promising results like these:

Figure 3: The reference dog image

Figure 4: The reference cat image

Figure 5: The reference teapot image

Figure 6: Prompt: An image of a dog and cat in space

Figure 7: Prompt: An image of a cat and a teapot

Figure 8: Prompt: An image of a cat and a teapot next to it

9 Conclusion

We extended SISO to the multisubject image personalization domain. We show that SISO is capable of producing high-fidelity images given multiple reference photos after iterating the fine-tuning through each image. We first attempted a masked-based approach to multiple reference image personalization which failed to generalize to distinct subjects. Implementing an inverse mask-based method to implement multi-subject finetuning to improve this still led to poor results and high memory usage. We found the iterative method worked to generalize to multiple subjects. Future work to improve this include testing on more than two subjects in one image and reducing memory constraints to allow for more nuanced image personalization.

References

[1] Zecheng He et al. "Imagine yourself: Tuning-Free Personalized Image Generation". In: *Meta* (2024). URL: https://ai.meta.com/research/publications/imagine-yourself-tuning-free-personalized-image-generation/.

[2] Mathilde Caron et al. "Emerging Properties in Self-Supervised Vision Transformers". In: *arXiv preprint arxiv:2104.14294* (2021).

[3] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arxiv:2106.09685* (2021). arXiv: 2106.09685 [cs.CL]. URL: https://arxiv.org/abs/2106.09685.

[4] Alexander Kirillov et al. "Segment Anything". In: *arXiv preprint arXiv:2304.02643* (2023).

[5] Zhen Li et al. *PhotoMaker: Customizing Realistic Human Photos via Stacked ID Embedding*. 2023. arXiv: 2312.04461 [cs.CV]. URL: https://arxiv.org/abs/2312.04461.

[6] Shilong Liu et al. "Grounding DINO: Marrying DINO with Grounded Pre-Training". In: *arXiv preprint arXiv:2303.05499* (2023).

[7] Or Patashnik et al. *Nested Attention: Semantic-aware Attention Values for Concept Personalization*. 2025. arXiv: 2501.01407 [cs.CV]. URL: https://arxiv.org/abs/2501.01407.

[8] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: https://arxiv.org/abs/2112.10752.

[9] Nataniel Ruiz et al. "DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation". In: *arXiv preprint arXiv:2208.12242* (2022).

[10] Shihao Shao and Qinghua Cui. "1st Place Solution in Google Universal Images Embedding". In: *arXiv preprint arxiv:2210.08473* (2022). arXiv: 2210.08473 [cs.CV]. URL: https://arxiv.org/abs/2210.08473.

[11] Stability AI. "SDXL Turbo: Ultra-Fast Text-to-Image Generation". In: (2023).

[12] Qixun Wang et al. "InstantID: Zero-shot Identity-Preserving Generation in Seconds". In: *arXiv preprint arxiv:2401.07519* (2024). arXiv: 2401.07519 [cs.CV]. URL: https://arxiv.org/abs/2401.07519.

[13] Yuxiang Wei et al. *ELITE: Encoding Visual Concepts into Textual Embeddings for Customized Text-to-Image Generation*. 2023. arXiv: 2302.13848 [cs.CV]. URL: https://arxiv.org/abs/2302.13848.

[14] Guangxuan Xiao et al. "FastComposer: Tuning-Free Multi-Subject Image Generation with Localized Attention". In: *arXiv preprint arxiv:2305.10431* (2023). arXiv: 2305.10431 [cs.CV]. URL: https://arxiv.org/abs/2305.10431.

[15] Shitao Xiao et al. *OmniGen: Unified Image Generation*. 2024. arXiv: 2409.11340 [cs.CV]. URL: https://arxiv.org/abs/2409.11340.

[16] Enze Xie et al. "SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers". In: *arXiv preprint arXiv:2410.10629* (2024).

[17] Idan Schwartz Yair Shpitzer Gal Chechik. "Single Image Iterative Subject-driven Generation and Editing". In: *arXiv preprint arxiv:2503.16025* (2025).

[18] Hu Ye et al. "IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models". In: *arXiv preprint arxiv:2308.06721* (2023). arXiv: 2308.06721 [cs.CV]. URL: https://arxiv.org/abs/2308.06721.

[19] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. "Adding Conditional Control to Text-to-Image Diffusion Models". In: *arXiv preprint arxiv:2302.05543* (2023). arXiv: 2302.05543 [cs.CV]. URL: https://arxiv.org/abs/2302.05543.

Appendix

A) Multi Subject Mask

Image Generated in the first iteration.

Failed Mask (Both Masks are same)

Figure 9: Trying to get the masks of the generated image using Grounding DINO and SAM. For both subjects - cat and teapot ; we get the same mask.

B) SISO Code Snippet

```
accelerator = Accelerator(gradient_accumulation_steps=gradient_accumulation_steps)
model_path = "stabilityai/sdxl-turbo"
"""
  Refer to
    https://huggingface.co/docs/diffusers/en/training/lora
    https://github.com/huggingface/diffusers/blob/e4b8f173b97731686e290b2eb98e7f5df2b1b322/examples/text_to_image/train_text_to_image_lora.py

"""

noise_scheduler = DDPMScheduler.from_pretrained(model_path, subfolder="scheduler")
tokenizer = CLIPTokenizer.from_pretrained(model_path, subfolder="tokenizer")
text_encoder = CLIPTextModel.from_pretrained(model_path, subfolder="text_encoder")
text_encoder_2 = CLIPTextModelWithProjection.from_pretrained(model_path, subfolder="text_encoder_2")
unet = UNet2DConditionModel.from_pretrained(model_path, subfolder="unet")
vae = AutoencoderKL.from_pretrained(model_path, subfolder="vae")

vae.requires_grad_(False)
text_encoder.requires_grad_(False)
text_encoder_2.requires_grad_(False)
unet.requires_grad_(False)

lora_config = LoraConfig(
    r=LORA_RANK,
    lora_alpha=LORA_ALPHA,
    init_lora_weights="gaussian",
    target_modules=["to_k", "to_q", "to_v", "to_out.0"],
)

unet.add_adapter(lora_config)
lora_layers = list(filter(lambda p: p.requires_grad, unet.parameters()))

optimizer = torch.optim.AdamW(
    lora_layers,
    lr=LR,
    betas=ADAM_BETAS,
    weight_decay=ADAM_WEIGHT_DECAY,
    eps=ADAM_EPSILON
)

lr_scheduler = get_scheduler(
    lr_scheduler_type,
    optimizer=optimizer,
    num_warmup_steps=lr_warmup_steps * gradient_accumulation_steps,
    num_training_steps=num_train_epochs,
)
unet, optimizer, lr_scheduler = accelerator.prepare(unet, optimizer, lr_scheduler)
```

Figure 10: Sequential optimization steps for each subject.

```python
# LPIPS
def LPIPS_loss(reference_images, generated_images):
  assert len(reference_images.shape) == 4, "Reference images must have dimensions (B, C, H, W)"
  assert len(generated_images.shape) == 4, "Generated images must have dimensions (B, C, H, W)"
  assert reference_images.shape[0] > 1, "Reference images must have more than 1 example"
  assert generated_images.shape[0] > 1, "Reference images must have more than 1 example"
  lpips = LearnedPerceptualImagePatchSimilarity(net_type='alex').to(device)
  return lpips(reference_images, generated_images).item()


# FID
def FID_score(reference_images, generated_images):
  assert len(reference_images.shape) == 4, "Reference images must have dimensions (B, C, H, W)"
  assert len(generated_images.shape) == 4, "Generated images must have dimensions (B, C, H, W)"
  assert reference_images.shape[0] > 1, "Reference images must have more than 1 example"
  assert generated_images.shape[0] > 1, "Reference images must have more than 1 example"
  fid = FrechetInceptionDistance(normalize=True).to(device)
  fid.update(reference_images, real=True)
  fid.update(generated_images, real=False)
  return float(fid.compute())

# KID
def KID_score(reference_images, generated_images):
  assert len(reference_images.shape) == 4, "Reference images must have dimensions (B, C, H, W)"
  assert len(generated_images.shape) == 4, "Generated images must have dimensions (B, C, H, W)"
  assert reference_images.shape[0] > 1, "Reference images must have more than 1 example"
  assert generated_images.shape[0] > 1, "Reference images must have more than 1 example"
  kid = KernelInceptionDistance(subset_size=2, normalize=True).to(device) # TODO: Look into subset_size
  kid.update(reference_images, real=True)
  kid.update(generated_images, real=False)
  return kid.compute()[0], kid.compute()[1]
```

Figure 11: Sequential optimization steps for each subject.